

Practical 3 – Problem sheet

Exercise 1

In this exercise, you will once more compute sums and similar expressions using loops; see Exercise 5 from Practical 2 for similar examples.

However, as an additional task, the function declarations are now no longer given, and you need to add them on your own. The function names you use should be `sumA` for part (a), `sumB` for part (b), etc., unless specified otherwise. The parameters and return types are specified in the individual parts below; the return value should be the value of the sum (or product) in question, and its type will either be specified, or will be clear from the context.

- (a) (S) Compute $\sum_{k=1}^n (k^2 - 1)$, where n is given as an integer parameter. Use integer arithmetic throughout. That is, the function `sumA` should take one `int` value as its parameter, and have an `int` return type.
- (b) (S) Compute $\sum_{j=3}^n \frac{j}{2j-5}$, where the integer $n \geq 3$ is given as a parameter. Use floating point arithmetic, i.e., the `double` datatype, also for the return value.
- (c) (S) Compute $\sum_{k=0}^m e^{kx/2}$, where the integer $m \geq 0$ and the floating point number x are given as parameters.
- (d) (I) Compute $\prod_{k=1}^n (k^2 + 2)$, where the integer $n \geq 1$ is given as a parameter. Use long integer arithmetic. The function name should be `productD`.
- (e) (I) Compute $\sum_{k=0}^n \sum_{j=0}^k \frac{1}{1 + aj^2 + bk^2}$, where the integer $n \geq 0$ and floating point numbers $a, b \geq 0$ are given as parameters.
- (f) (I) For $n \in \mathbb{N}$, let $A_n := \{2, 3, 4, 5, 8, 9, \dots, 2^n, 2^n + 1\}$. Compute $\sum_{a \in A_n} \sqrt{a}$, given $n \geq 1$ as a parameter.
- (g) (A) Given an integer $n \geq 1$, compute $\sum_{\substack{p \text{ prime} \\ p \leq n}} \frac{1}{p}$.
- (h) (A) Functions in Java can have a variable number of arguments (“varargs”). Read about this language feature. Then, as an application, write a function `sumH` that takes several floating point numbers $a_1, \dots, a_n$ as arguments, and computes $\sum_j a_j^{-2}$. In order to check your work, write another function (with name of your own choice) that invokes the previous one for the numbers 2.0, 0.1, 10.0.

Exercise 2

In this exercise, we use loops to compute certain integer sequences. Since some of these grow very fast, let us use the `long` integer type for their values.

Write functions `sequenceA`, `sequenceB`, etc., that take an integer n (`int`) as their input – and possibly extra arguments as specified individually below – and that return a_n , b_n , etc.

(S) $a_1 = 2$, $a_{n+1} = -2a_n + 7$ for $n \geq 1$.

(S) $b_0 = 1$, $b_{n+1} = b_n^2 - 6nb_n$ for $n \geq 0$.

(I) $c_0 = \alpha$, $c_1 = \beta$, $c_{n+1} = 2c_n - 3c_{n-1}$ for $n \geq 1$, where α, β are given as long integer parameters.

(I) $d_1 = \gamma$, $d_{n+1} = \begin{cases} d_n^2 + 2 \bmod k & \text{if } n \geq 1 \text{ is even,} \\ d_n^3 - 3 \bmod k & \text{if } n \geq 1 \text{ is odd.} \end{cases}$

Here $\gamma \in \mathbb{Z}$ and $k \geq 2$ are given as long integer parameters. Regarding division remainders, we follow the conventions from the Algebra module, i.e., $0 \leq d_n < k$ for all $n > 1$.

(I) $e_0 = 1$, $e_n = \frac{2(2n-1)}{n+1}e_{n-1}$ ($n > 0$). These are known as the *Catalan numbers*.

Hint: While the Catalan numbers are always integers, the fraction $\frac{2(2n-1)}{n+1}$ is not always an integer – be sure to perform the division *after* all relevant multiplications.

Exercise 3

For evaluating trigonometric functions, we are used to rely on our pocket calculator or (in this module) on Java's built-in `Math` class, which give a numeric approximation of these functions to a certain precision. In this exercise, we want to compute a numeric approximation for the sine function “from first principles”.

(a) (S) Preparatory task: Write a function that computes the factorial $n!$ of an integer n . The name of the function should be `factorial`, it should have one argument of type `int`, and its output value should be of type `double`. (You can take a similar function introduced in Lesson 10 as a guideline; but make sure that the computation is done with `double` floating point numbers rather than integers.) You may assume the function argument is a positive number.

(b) (I) A good way to compute $\sin(x)$ on the interval $[0, 2\pi]$ is by its power series,

$$\sin(x) = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} x^{2k+1}. \quad (1)$$

We approximate this expression by replacing the infinite series $\sum_{k=0}^{\infty}$ with a finite sum $\sum_{k=0}^n$, where n is chosen such that the last term in the sum (with index $k = n$) is the first one to be smaller than a certain cutoff ϵ in absolute value. That is, n is the smallest index such that

$$\left| \frac{(-1)^n}{(2n+1)!} x^{2n+1} \right| < \epsilon. \quad (2)$$

Write a function `sine` that takes two arguments:

- the point $x \in [0, 2\pi]$, of type `double`,
- the cutoff ϵ , of type `double`,

and produces an approximation of $\sin(x)$ using the algorithm outlined above, with return value of type `double`.

Also check the output of your function for x outside the interval $[0, 2\pi]$, in particular, for large positive x . What do you notice? What might be the reason for the behaviour you observe?

(c) (I) For arbitrary x , one can approximate $\sin(x)$ by using its periodicity property: One chooses $n \in \mathbb{Z}$ such that $x' = x - 2\pi n \in [0, 2\pi]$, and then computes $\sin(x) = \sin(x')$ by the algorithm in b).

Write a function `sineLargeDomain`, with the same parameters and output type as `sine` above, that computes $\sin(x)$ for arbitrary x by this algorithm. It is permitted (in fact, it is good programming style) to call the function `sine` from above within the code for `sineLargeDomain`.

Hint for testing: Three different unit tests are provided for parts (a), (b) and (c). The relevant test will compile only after the relevant functions in (a), (b), (c) have been implemented.

For testing “by hand”, a good idea is to choose values of x for which $\sin(x)$ is explicitly known.

Exercise 4 (I)

Consider a smooth function f on an interval $[a, b]$. The integral $\int_a^b f(x) dx$ corresponds to the area under the graph of f . Let us approximate this area with a trapezium through the points $(a, 0)$, $(a, f(a))$, $(b, f(b))$, $(b, 0)$. By elementary geometry, the area of this trapezium is $A = \frac{1}{2}(f(a) + f(b))(b - a)$. Thus an approximation for the integral is

$$\int_a^b f(x) dx \approx \frac{(b - a)}{2} (f(a) + f(b)). \quad (3)$$

This is called the *Trapezium rule*.

The approximation is quite rough if f varies over the interval. It can be improved by splitting the interval into n equal pieces of length $h = (b - a)/n$, and approximating the integral by a trapezium area on each subinterval. That leads to*

$$\int_a^b f(x) dx \approx \frac{h}{2} \left(f(x_0) + 2 \sum_{j=1}^{n-1} f(x_j) + f(x_n) \right), \quad \text{where } x_j = a + jh. \quad (4)$$

This is called the *Composite Trapezium rule*.

- Let $f(x) = \exp(-3x^2)$. Implement a Java function `f` that evaluates f , taking `double` numbers as its (only) argument and as its output value.
- Write a function `trapeziumInt` that computes $\int_a^b f(x) dx$ by the Trapezium rule, with f as above. It should take a and b as parameters (type `double`) and produce a `double` as output.

*You might want to do this computation if you're interested, but it's not part of this exercise. For further details, see, e.g., Burden and Faires, *Numerical Analysis*, Sec. 4.3 and 4.4.

- (c) Write a function `compositeTrapeziumInt` that computes $\int_a^b f(x) dx$ by the Composite Trapezium rule, with f as above. It should take a and b (type `double`) and n (type `int`) as parameters, and produce a `double` as output.

In (b) and (c), use the function implemented in (a) whenever appropriate.

For testing, you may want to compare your results against values of $\int_a^b f(x) dx$ obtained with Maple, Wolfram Alpha, or similar.

Exercise 5 A

In extension of the previous exercise, let us now look for a way to numerically approximate the double integral

$$\int_{-a}^a dx \int_{-a}^a dy \log(x^2 y^2 + 1) e^{-x^2 - y^2}, \quad (5)$$

where $a > 0$ is a constant.

There is a generalization of the (composite) Trapezium rule for two-dimensional integration which can be used to evaluate the integral (5). But there are many other numerical integration schemes for the same purpose,[†] such as the *two-dimensional Simpson's rule*.

Read about these integration methods. Then, implement a function `trapeziumInt2d()` that evaluates (5) using the two-dimensional Trapezium rule. It should take two arguments: The number a (`double`, assumed positive), and the number of subintervals n in the composite integration rule (`int`, assumed positive). Here you should divide both the integration interval on the x axis and on the y axis into n steps each. Then, write an analogous function `simpsonInt2d()` which uses the two-dimensional (composite) Simpson's rule instead.

Compare the accuracy two methods by computing their approximation errors for moderate n (say, around 100). For reasons of “fairness” of the comparison, you should choose the number of subintervals such that both methods use roughly the same number of evaluations of the integrand. (For comparing the errors, you need to know an “exact” value of the integral; for this, take the output of `trapeziumInt2d()` or `simpsonInt2d()` for sufficiently large n .)

Note: Unit tests are provided only for the Trapezium rule.

[†]This is true for the one-dimensional case as well, we just did not discuss it.